

Mixture-of-Experts Top-K Routing, Worked Out by Hand

KEEP ALL THE PARAMETERS; USE A FRACTION PER TOKEN — ONE TOKEN, EIGHT EXPERTS, TOP-2

What this document does. It routes *one* token through a layer of eight experts with top-2 routing, entirely by hand. We score the experts with a softmax router, pick the top two, renormalize their gates, form the weighted-sum output, and tally exactly how many parameters were “active” for this token versus how many the layer holds. Every number is produced by a short NumPy script (Appendix A, `m06_t04_moe_gating_engine.py`).

The one idea. A dense FFN sends every token through *all* its parameters. An MoE splits that FFN into N experts and sends each token through only the top $K \ll N$. Total capacity (knowledge) stays high; active compute and — crucially for memory-bound decode — active *bytes read* per token drop to a fraction. You pay for breadth once and rent only a slice per token.

CONCEPTS COVERED, IN ORDER

the router $s = \text{softmax}(W_g h)$; top- K selection; gate renormalization $g_i = s_i / \sum_{j \in I} s_j$; the weighted-sum output $\sum_{i \in I} g_i E_i(h)$; shared vs. routed experts; and the active-parameter fraction that sets decode throughput.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The layer under study	2
2	Step 1 — The router softmax	2
3	Step 2 — Top- K selection	2
4	Step 3 — Renormalize the gates	3
5	Step 4 — The weighted-sum output	3
6	Step 5 — Active vs. total parameters	4
7	The argument, at a glance	5
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The layer under study

DeepSeek-V3 (256 experts, top-8) and Mixtral (8 experts, top-2) are production MoEs. We use the Mixtral shape — eight experts, top-2 — and follow one token.

Quantity	Symbol	Value here
Number of experts	N	8
Experts per token	K	2 (top-2)
Router logits $W_g h$	—	(0.5, −1.0, 2.2, 0.1, −0.4, 1.6, 0.3, −0.8)
Hidden width	d	5120
Shared experts (Nexus)	M	2 always-on

Intuition

Think of the eight experts as eight specialists in a consulting firm and the router as the receptionist. A client (token) walks in; the receptionist glances at the request and forwards it to the two most relevant specialists — not all eight. The firm *employs* eight experts' worth of knowledge, but any one client only *occupies* two. That is the entire economic trick of MoE.

2 Step 1 — The router softmax

The token's hidden state $h \in \mathbb{R}^d$ hits a small linear router $W_g \in \mathbb{R}^{N \times d}$, producing one logit per expert, then a softmax:

Router scores

$$s = \text{softmax}(W_g h) \in \mathbb{R}^N, \quad s_i = \frac{\exp((W_g h)_i)}{\sum_{j=1}^N \exp((W_g h)_j)}.$$

From the logits above:

$$s = (0.0843, 0.0188, \mathbf{0.4612}, 0.0565, 0.0343, \mathbf{0.2531}, 0.0690, 0.0230),$$

which sums to 1. Expert 3 ($s_3 = 0.461$) and expert 6 ($s_6 = 0.253$) clearly dominate; the other six split the remaining $\sim 29\%$.

3 Step 2 — Top- K selection

Keep only the $K = 2$ highest-scoring experts and discard the rest for this token:

$$I = \text{topk}(s, 2) = \{3, 6\} \subset \{1, \dots, 8\}.$$

Experts $\{1, 2, 4, 5, 7, 8\}$ stay *idle* — their FFNs are never evaluated, their weights never read. This is the sparsity that makes MoE cheap: 6 of 8 experts contribute nothing to this token's forward pass.

Watch out

Top- K is a hard, non-differentiable choice (an arg operation). Gradients flow only through the K selected experts and through the gate *values* of those experts — never through the

experts that were not picked. That is precisely why MoEs need an auxiliary load-balance loss (Topic 5): nothing in the main loss encourages the router to ever *try* a neglected expert.

4 Step 3 — Renormalize the gates

The two surviving scores (0.461 and 0.253) do not sum to 1 — they summed to 1 only *across all eight*. To get convex combination weights we renormalize over the selected set:

Gate renormalization

$$g_i = \frac{s_i}{\sum_{j \in I} s_j} \quad \text{for } i \in I, \quad g_i = 0 \text{ otherwise.}$$

Compute the two gates

The selected-set normalizer is

$$\sum_{j \in I} s_j = s_3 + s_6 = 0.4612 + 0.2531 = 0.7143.$$

Therefore

$$g_3 = \frac{0.4612}{0.7143} = 0.6457, \quad g_6 = \frac{0.2531}{0.7143} = 0.3543, \quad g_3 + g_6 = 1. \quad \checkmark$$

The full gate vector is

$$g = (0, 0, \mathbf{0.6457}, 0, 0, \mathbf{0.3543}, 0, 0).$$

Expert 3 gets about a 65% vote, expert 6 about 35%.

5 Step 4 — The weighted-sum output

Only the selected experts run, and their outputs are blended by the gates:

MoE layer output

$$o = \sum_{i \in I} g_i E_i(h) = g_3 E_3(h) + g_6 E_6(h).$$

With the two experts evaluated on $h = (1.0, -0.5, 0.5, 0.25)$:

$$E_3(h) = (-0.116, 0.088, -0.518, 0.675),$$

$$E_6(h) = (0.022, -0.454, -0.588, 0.158),$$

$$o = 0.6457 E_3(h) + 0.3543 E_6(h) = (-0.067, -0.104, -0.543, 0.492). \quad \checkmark$$

That four-vector is the MoE layer's output for this token. The other six experts never touched it.

Intuition

The gates are a learned, per-token mixing ratio. A token that is 90% “code” and 10% “prose” might come out $g = (0.9, 0.1)$ on a code expert and a prose expert. The output smoothly interpolates between specialists — and because the weights are re-derived every token, the same layer behaves like a different sub-network for every input.

6 Step 5 — Active vs. total parameters

This is the payoff metric. Each expert is a full FFN; a SwiGLU-style FFN is $\approx 3 d_{\text{model}} d_{\text{ffn}}$ parameters. With $d_{\text{model}} = 5120$, $d_{\text{ffn}} = 3584$:

$$\text{per-expert} = 3 \times 5120 \times 3584 = 55,050,240 \text{ params.}$$

Nexus uses 8 routed + 2 shared (always-on) experts. So per MoE layer:

	Experts counted	Params
Total (all experts resident)	$8 + 2 = 10$	550,502,400
Active per token (top-2 + 2 shared)	$2 + 2 = 4$	220,200,960

Active-parameter fraction

$$\text{active fraction} = \frac{(K + M) \cdot p_{\text{exp}}}{(N + M) \cdot p_{\text{exp}}} = \frac{2 + 2}{8 + 2} = \frac{4}{10} = 0.40.$$

Only 40% of the layer’s FFN parameters are touched per token — a $2.5\times$ reduction in active FFN params versus reading all ten experts. Because decode is memory-bandwidth-bound (Module 01), reading $\sim 40\%$ of the bytes is $\sim 2.5\times$ fewer bytes per token, which on the FFN portion translates almost directly into throughput.

Capacity stays, compute shrinks

The model still *stores* all ten experts (550M params/layer of knowledge), but any single token *activates* only four (220M). At Nexus v6 scale this is the 70B-total / 38B-active split, $\approx 1.84\times$ throughput on memory-bound decode — the same byte-counting argument from Module 01, now applied per expert instead of per layer.

Watch out

The active fraction governs *compute and bandwidth*, not *memory residency*. All ten experts must still live in GPU (or pooled) memory, because any token might route to any of them. MoE trades memory footprint (you hold everything) for compute/bandwidth per token (you use a slice). When the experts no longer fit on one GPU you shard them — which is the all-to-all story of Topic 5.

7 The argument, at a glance

#	Step	Result
1	Router	$s = \text{softmax}(W_g h)$; $s_3=.461$, $s_6=.253$
2	Top- K	$I = \{3, 6\}$; six experts idle
3	Renorm	$g_3=0.646$, $g_6=0.354$ (sum 1)
4	Output	$o = g_3 E_3(h) + g_6 E_6(h)$ ✓
5	Active frac	$(K+M)/(N+M) = 4/10 = 40\%$

7.1 Equation cheat-sheet

Router scores	$s = \text{softmax}(W_g h)$
Selection	$I = \text{topk}(s, K)$
Gate renorm	$g_i = s_i / \sum_{j \in I} s_j$, $i \in I$
Layer output	$o = \sum_{i \in I} g_i E_i(h)$
Active fraction	$(K + M) / (N + M)$

Appendix A — Reference implementation

Every number above is produced by `m06_t04_moe_gating_engine.py`. Run it to reproduce the router softmax, the top-2 gates, the weighted-sum output, and the active-parameter tally; change the logits or K to explore other routings.

```

1  """
2  Reference implementation for: Module 06 - Topic 4
3  "Mixture-of-Experts top-K routing - gates, weighted sum, active parameters."
4
5  Route ONE token through 8 experts with top-2 routing. We compute the router
6  softmax, pick the top-2 experts, renormalize their gates, form the weighted-sum
7  output, and tally the active-vs-total parameter fraction.
8  """
9
10 import numpy as np
11 np.set_printoptions(precision=4, suppress=True)
12
13 def softmax(z):
14     z = np.asarray(z, dtype=float)
15     z = z - z.max()
16     e = np.exp(z)
17     return e / e.sum()
18
19 # -----
20 # 0. One token's router logits over N=8 experts; top_k=2.
21 #   Logits = W_g @ h ; we give the resulting logit vector directly.
22 # -----
23 N = 8
24 K = 2
25 gate_logits = np.array([0.5, -1.0, 2.2, 0.1, -0.4, 1.6, 0.3, -0.8]) # W_g h
26 print("== 0. Router logits (W_g h) over 8 experts ==")
27 print("logits =", gate_logits)
28
29 # -----
30 # 1. Softmax over all experts.
31 # -----
32 s = softmax(gate_logits)

```

```

33 print("\n== 1. s = softmax(W_g h) ==")
34 print("s =", s)
35 print("sum(s) =", s.sum())
36
37 # -----
38 # 2. Top-K selection.
39 # -----
40 topk_idx = np.argsort(s)[-K:][::-1]          # indices of top-2, descending
41 topk_idx_sorted = np.sort(topk_idx)
42 print(f"\n== 2. Top-{K} experts by score ==")
43 print("chosen expert indices (0-based) =", topk_idx_sorted,
44       " -> experts", topk_idx_sorted + 1, "(1-based)")
45 print("their raw softmax scores =", s[topk_idx_sorted])
46
47 # -----
48 # 3. Renormalize gates over the selected set.
49 #   g_i = s_i / sum_{j in I} s_j   for i in I, else 0.
50 # -----
51 sel = np.zeros(N, dtype=bool)
52 sel[topk_idx] = True
53 denom = s[sel].sum()
54 g = np.zeros(N)
55 g[sel] = s[sel] / denom
56 print("\n== 3. Renormalized gates g_i = s_i / sum_{j in I} s_j ==")
57 print("denominator sum_{j in I} s_j =", denom)
58 print("g =", g)
59 print("active gates:", {int(i+1): round(float(g[i]), 4) for i in np.where(sel)[0]})
60 print("sum of active gates =", g.sum())
61
62 # -----
63 # 4. Output = sum_{i in I} g_i * Expert_i(h).
64 #   Mock each expert as a simple deterministic function of a 4-dim h so the
65 #   weighted sum is a concrete vector.
66 # -----
67 h = np.array([1.0, -0.5, 0.5, 0.25])
68 rng = np.random.default_rng(0)
69 # Fixed expert "weight" matrices [4x4] so Expert_i(h) is reproducible.
70 experts = [rng.normal(0, 0.5, size=(4, 4)) for _ in range(N)]
71 def expert_fwd(i, h):
72     return np.tanh(experts[i] @ h)
73
74 out = np.zeros(4)
75 print("\n== 4. Weighted-sum output ==")
76 for i in np.where(sel)[0]:
77     ei = expert_fwd(i, h)
78     out += g[i] * ei
79     print(f"  expert {i+1}: g={g[i]:.4f}  Expert(h)={ei}")
80 print("output o = sum g_i Expert_i(h) =", out)
81
82 # -----
83 # 5. Active-parameter fraction.
84 #   Total FFN params = N experts x per_expert. Active = K experts x per_expert.
85 #   Add shared experts (always on) and the router for a realistic tally.
86 # -----
87 print("\n== 5. Active-parameter accounting ==")
88 d_model, d_ffn = 5120, 3584
89 # A SwiGLU-style FFN ~ 3 matrices of d_model x d_ffn.
90 per_expert = 3 * d_model * d_ffn
91 n_routed, n_shared = 8, 2
92 total_ffn = (n_routed + n_shared) * per_expert
93 active_ffn = (K + n_shared) * per_expert      # top-2 routed + 2 shared
94 frac = active_ffn / total_ffn
95 print(f"per-expert FFN params      = {per_expert:,}")

```

```
96 print(f"total experts {n_routed}+{n_shared} shared = {total_ffn:},) params")
97 print(f"active per token: {K} routed + {n_shared} shared = {active_ffn:},) params")
98 print(f"active fraction = {active_ffn}/{total_ffn} = {frac:.3f}  ({frac*100:.1f}%)"
99 print(f"-> {1/frac:.2f}x fewer active FFN params per token than dense-equivalent")
100
101 # Headline Nexus-style numbers (illustrative, from the HTML).
102 print("\n== 5b. Nexus v6 headline (illustrative) ==")
103 total_B, active_B = 70.0, 38.0
104 print(f"total params {total_B}B, active per token {active_B}B "
105       f"-> {total_B/active_B:.2f}x throughput on memory-bound decode")
```

— END OF DOCUMENT —